

Design explanation

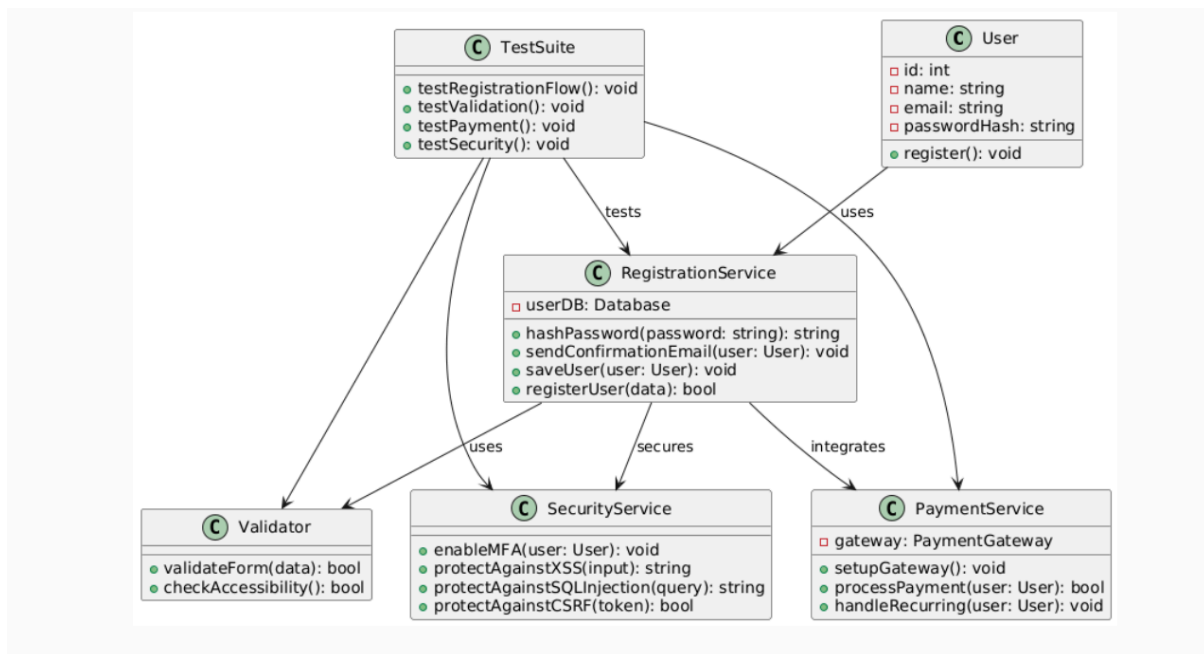
Interface

A minimal good enough design so that the group can quickly focus on functionality. There is a strong focus on user stories so the registration page is the home page. Colours are inspired by bright school colours and also for user engagement.

UML

We add good enough UML designs to only user stories that need them.

Purpose: Handles user registration, validation, security, and payment integration.



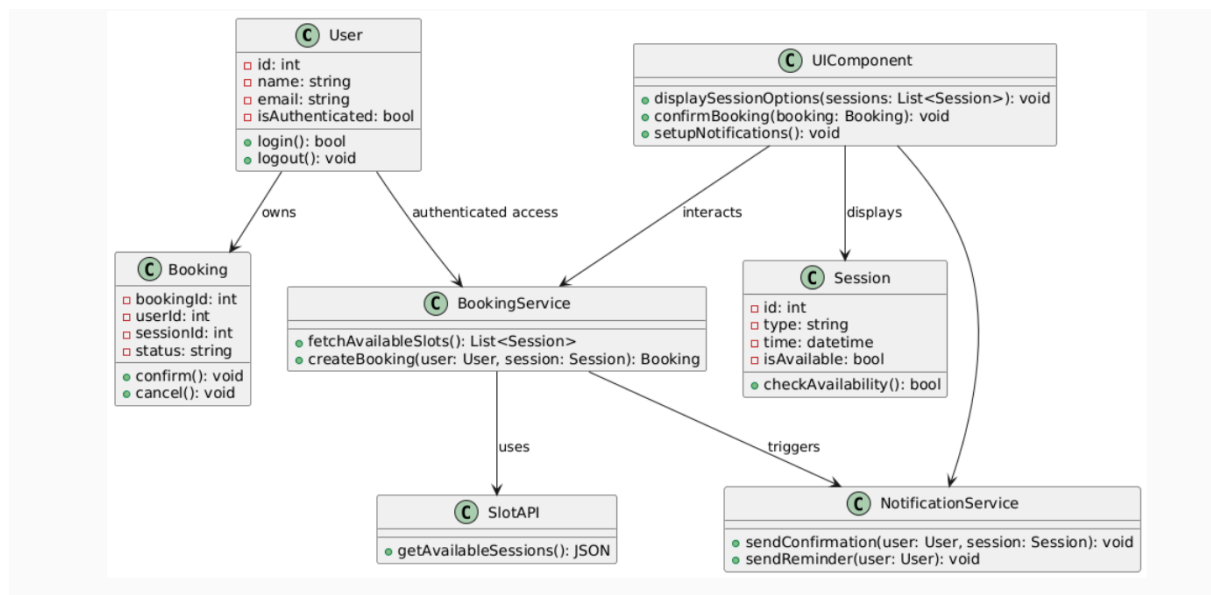
Key Components:

- **RegistrationService**: Core service that coordinates user registration.
 - Handles password hashing, email confirmation, saving users, and registration logic.
- **User**: Model storing user details (id, name, email, password hash).
- **Validator**: Validates form input and checks accessibility.
- **SecurityService**: Secures registration with XSS, SQLi, CSRF protection and MFA.
- **PaymentService**: Integrates payment handling and recurring billing.

- TestSuite: Contains unit tests for registration, validation, payment, and security.

Interactions:

- RegistrationService uses:
 - User to manage user data.
 - Validator to validate inputs.
 - SecurityService to secure the process.
 - PaymentService to integrate payment.
- TestSuite tests the whole flow.

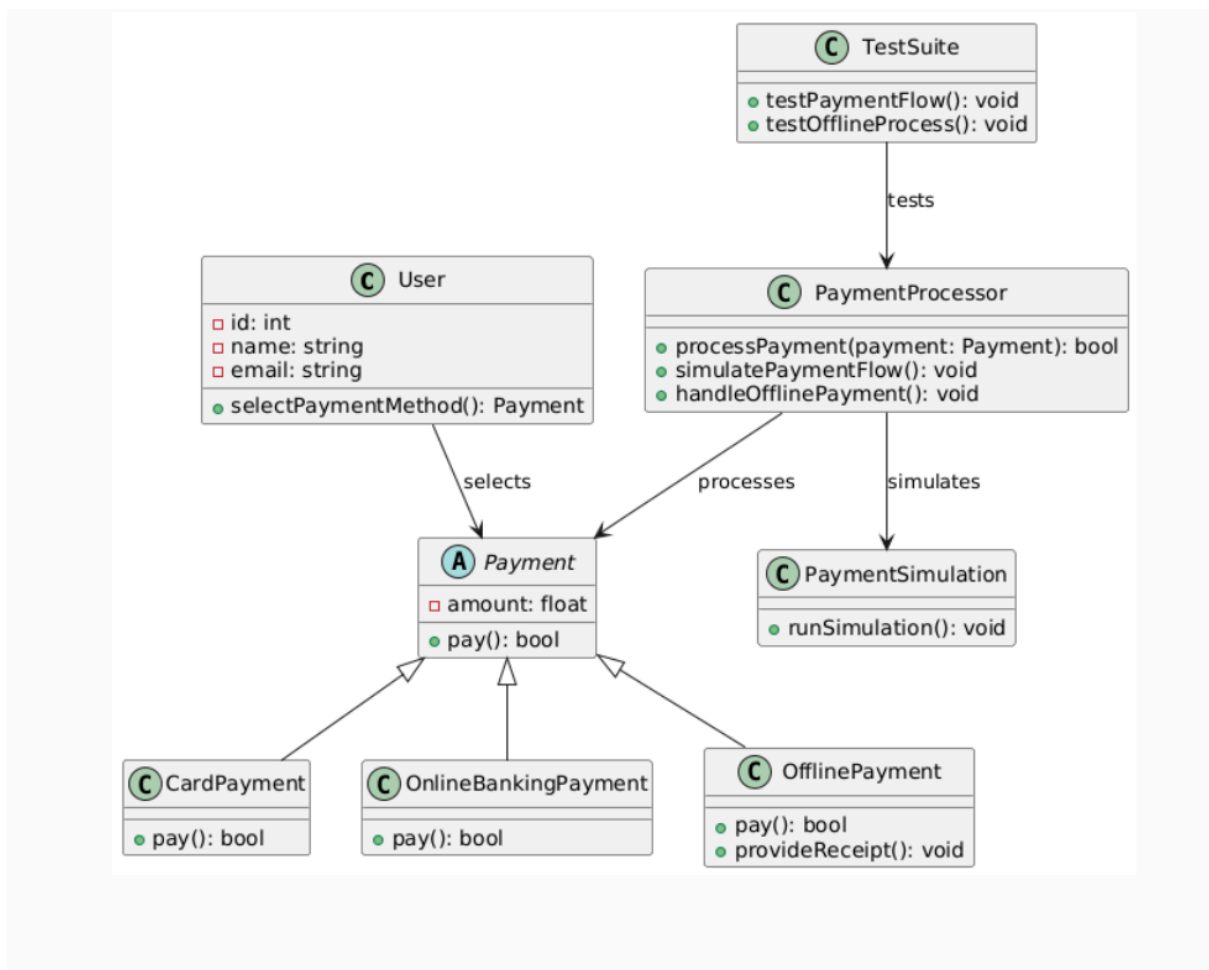


Purpose: Manages user bookings for time-based sessions with notifications and slot availability.

Key Components:

- User: Has login/logout, owns bookings.
- BookingService: Handles booking creation and available slots.
- Booking: Represents individual bookings (user ID, session ID, status).
- Session: Represents a session with ID, type, time, and availability.
- SlotAPI: Fetches available sessions (likely from external API).

- NotificationService: Sends confirmations and reminders.
 - UIComponent: Displays session options and booking feedback.
- Interactions:
- User books sessions through BookingService.
 - BookingService uses:
 - Session to check availability.
 - SlotAPI to fetch available slots.
 - NotificationService to send booking-related messages.
 - UIComponent interfaces with user for booking actions.



Purpose: Handles different payment methods (card, banking, offline) and testing via simulation.

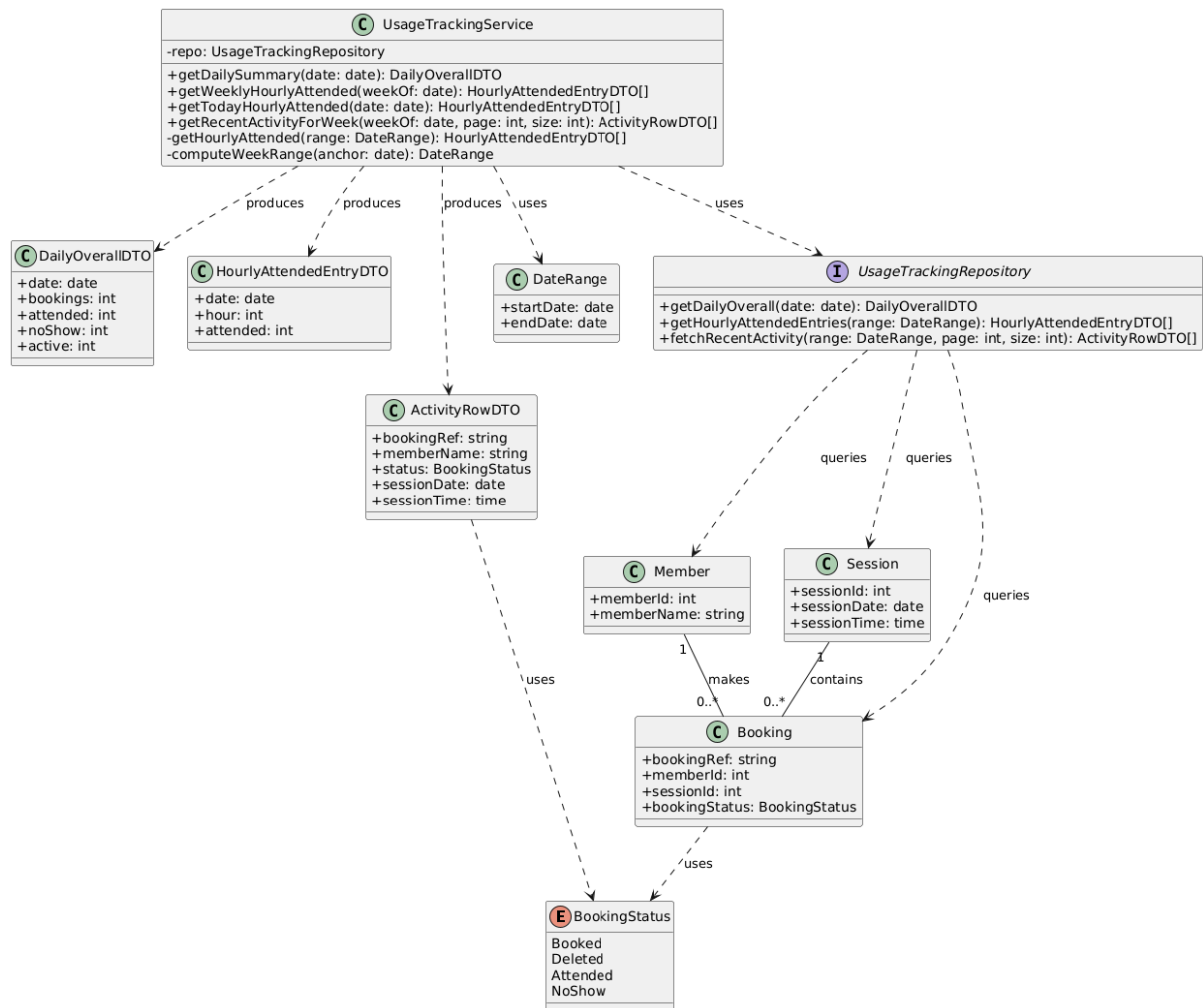
Key Components:

- User: Selects a payment method.

- Payment (abstract): Parent of all payment types, defines pay().
- CardPayment, OnlineBankingPayment, OfflinePayment: Concrete payment types that implement pay().
- PaymentProcessor: Processes payments and handles offline workflows.
- PaymentSimulation: Simulates payment flows for testing.
- TestSuite: Tests various payment flows.

Interactions:

- User selects a Payment type.
- PaymentProcessor:
 - Processes real or simulated payments.
- TestSuite ensures the payment logic works through testing.



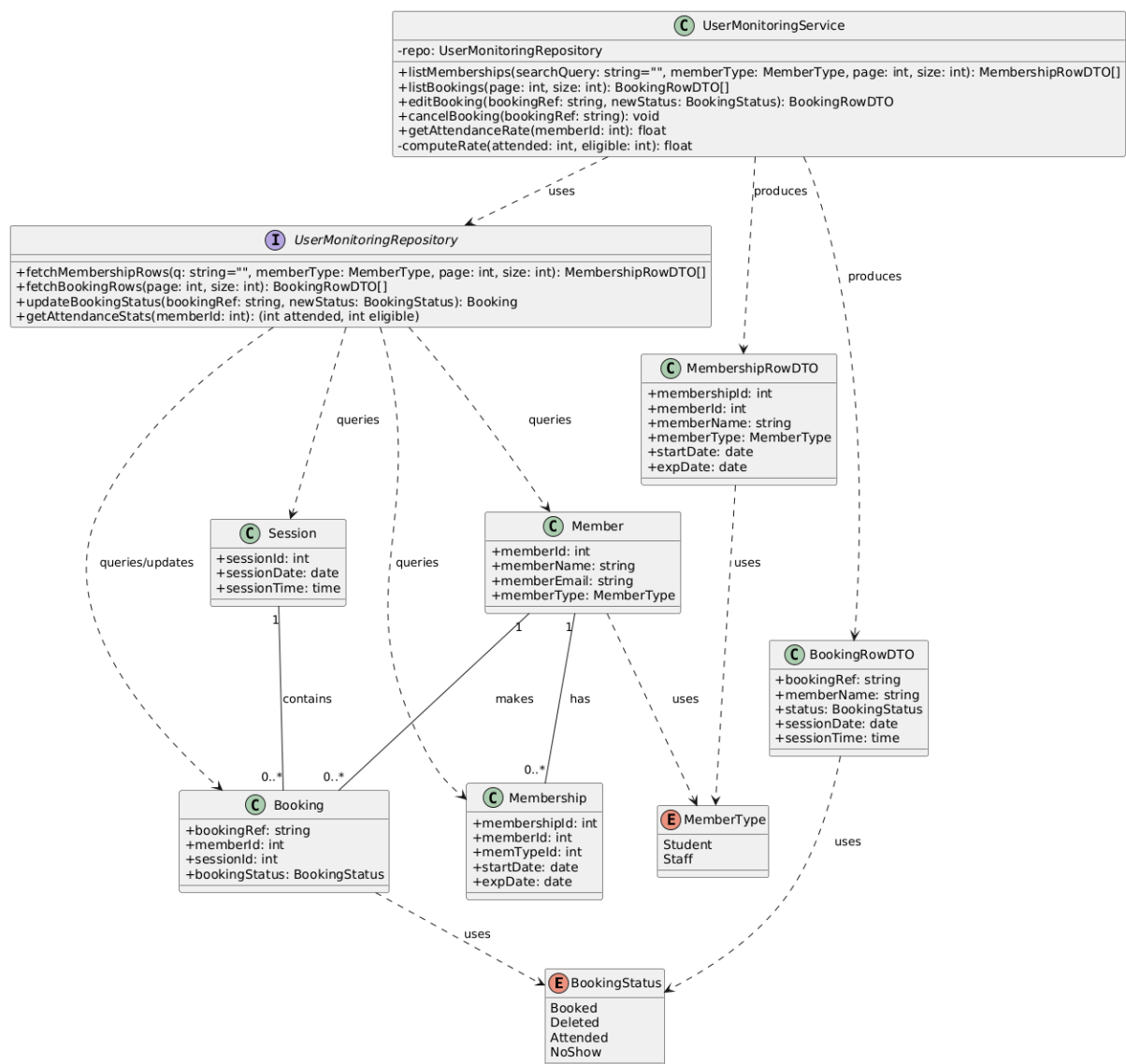
Purpose: Displays to administrators attendance and booking statistics from the database for a single selected date (and the week containing that date).

Key Components:

- **UsageTrackingService**: Orchestrates use cases; computes week ranges; delegates to the repository.
- **UsageTrackingRepository**: Runs DB queries over Booking, Session, and Member.
- **DailyOverallDTO**: Aggregated metrics for a given date (e.g., totals and counts by BookingStatus) returned to callers.
- **HourlyAttendedEntryDTO**: Series element used for bar charts.
- **ActivityRowDTO**: Row model for recent activity.
- **DateRange**: Start/end boundaries used by hourly and activity queries.
- **Domain entities**: Booking, Session, Member, and BookingStatus enum that the repository queries and maps into DTOs.

Interactions:

- Admin selects a date through UI.
- UsageTrackingService:
 - Computes range and fetches data via UsageTrackingRepository.



Purpose: Displays information from the database and allows admin actions like editing

Key Components:

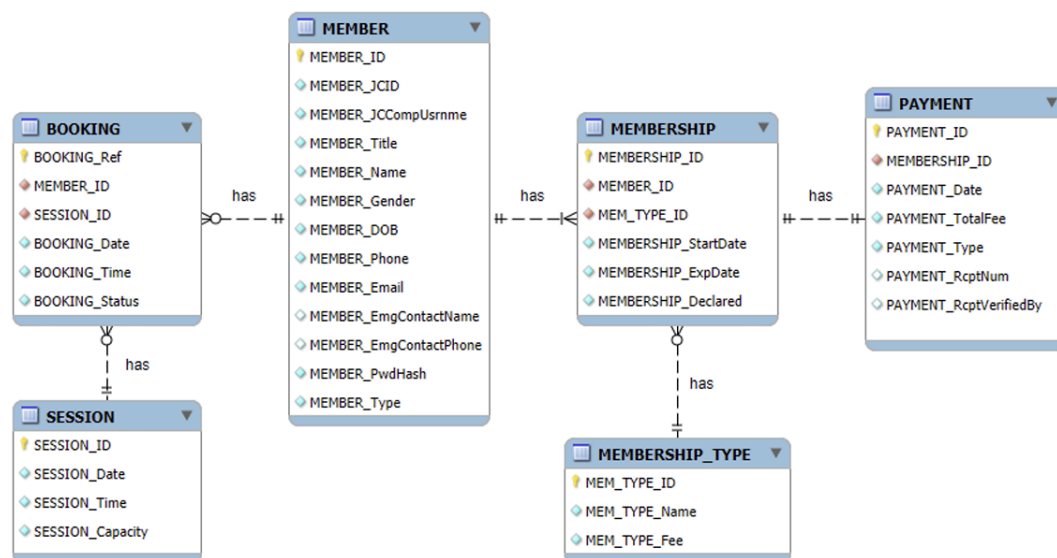
- UserMonitoringService: Lists memberships/bookings, edits/cancels bookings, computes attendance rate.
- UserMonitoringRepository: Fetches rows, updates booking status, provides attendance stats.
- MembershipRowDTO: Membership table row for the UI.
- BookingRowDTO: Booking table row for the UI.

- Domain entities: Member, Membership, Booking, Session; enums BookingStatus, MemberType.

Interactions:

- Admin searches for members, bookings, or views attendance rate.
- UserMonitoringService:
 - Retrieves DTO rows.
 - Updates booking status via UserMonitoringRepository.

ERD



(Please refer to the ERD design pdf for elaboration on the relationships and what is enforced by the web application)

The database is designed based on the existing system used by the school (e.g., registration form uses school ID and login uses computer username) and the principle of good enough design. It only covers the user stories and what the school uses within those (e.g., even if the existing system has fault reporting, the database does not cover that as it is not part of the user stories). It avoids overcomplication (e.g., instead of supertypes and subtypes, the database just uses a member_type). App enforcement is opted for if it makes the management system simpler.